Superr Mobile Automation — Senior QA Engineer Interview Preparation Guide

> ***Based entirely on real project work, framework development, debugging sessions, and architectural decisions from the Superr QA automation project.***
Target Role: Senior Mobile Automation Engineer (4-6 years experience)

Table of Contents

Section 1: Project Introduction

What is Superr?

Superr is an **educational tablet application** designed for students (Class 3 through Class 12, CBSE curriculum). It runs on custom Android tablets called the **Superr Book Tablet** and provides an interactive learning experience centered around handwriting, textbook browsing, quiz-based assessments, and AI-powered tutoring.

Main Modules

Module	Description	Key Screens
My Notes / Quick Notes	Digital notebook where students write with a stylus on a canvas	Notebook pages, pen palette, tool slot selection, page navigation
Library	Textbook shelf organized by class and subject	Book selection, CBSE class filter, chapter browser
Textbook Viewer	Read and annotate textbook pages with canvas overlay	Page canvas, AI Tutor panel, export, search
Quiz Arena	Gamified assessment engine with multiple quiz modes	Speed Run (timed), Classic, Survival, question types: MCQ, Fill-in-Blank, True/False

AI Tutor (SuperrChat)	AI assistant that reads handwritten content and explains it	Smart Markup, "Explain this", "Help me solve", "Quiz me", Chat History, canvas input
LMS Portal (Web)	Teacher-facing web portal for file uploads, class management	Login, Files section, upload/download, class selection
Admin Portal (Web)	Admin web portal for device management	Login, Device Management

Features Tested

- **Handwriting on canvas** — students write with a stylus; automation writes characters programmatically
- **Speed Run quiz** — timed quiz with mixed question types (MCQ, Fill-in-Blank, True/False)
- **Quiz result validation** — correct/incorrect/skipped counters, score screen
- **Leave Quiz flow** — cross button → dialog → redirect
- **AI Tutor interaction** — write text → Smart Markup → Explain this → AI response → follow-up chat
- **Library class switching** — change CBSE class selection from Class 1 through Class 12
- **File upload/delete** — LMS portal multi-file upload, right-click delete
- **Login flows** — session linking, blank field validation, invalid credentials, leading/trailing spaces

Platforms Supported

Platform	Details
Android Tablet	Superr Book Tablet (custom hardware), resolution 1200×1920
Web (LMS)	Chrome browser — teacher portal
Web (Admin)	Chrome browser — admin portal

Tech Stack

Layer	Technology
App Under Test	Android native application with Canvas-based custom views
Automation Framework	Java 17, Appium 2.x, TestNG, Maven
Mobile Driver	UiAutomator2
Web Driver	Selenium 4, WebDriverManager
Reporting	ExtentReports (HTML), Log4j2
Stylus Simulation	Custom StylusInjector APK (injects MotionEvents with TOOL_TYPE_STYLUS)
Handwriting Engine	Custom HandwritingEngine (glyph-based character drawing with normalized stroke coordinates)
Device Communication	ADB (Android Debug Bridge)

CI/CD	Maven command-line execution, configurable via TestNG XML suites

Application Architecture Classification

Verdict: Canvas-based Native Android app with Custom Views

Evidence from our project:

1. Canvas-based rendering: The notebook and quiz screens use Android Canvas for drawing. Elements behind the canvas are NOT inspectable by Appium — this was a recurring challenge. The Fill-in-Blank and True/False question types have a canvas overlay that completely hides the underlying UI elements (question text, question number, type labels).

2. Custom Views: The notebook is a custom view that handles stylus input, palm rejection, pen selection, and page management. It's not a standard Android EditText or WebView — it's a fully custom implementation.

3. Native Android: The app uses standard Android views (android.view.View, android.widget.TextView, android.widget.ImageView) alongside the custom canvas. The XML hierarchy from Appium Inspector shows native Android elements with `content-desc`, `text`, and `bounds` attributes — not web-based or Compose-based.

4. NOT Jetpack Compose: The element hierarchy shows traditional Android Views (`android.view.View`, `android.widget.TextView`), not Compose semantics. Compose elements would appear differently in the Appium Inspector (typically as ComposeView with semantic nodes). Our locators use `UiSelector()`, accessibility IDs, and XPath — all traditional Android automation patterns.

5. NOT Hybrid/WebView: No WebView containers were observed in the Inspector hierarchy. The AI Tutor panel, quiz screens, and notebook are all native Android views.

6. Mixed architecture: The notebook canvas coexists with standard Android views (buttons, text views, the pen palette). This hybrid of Canvas + traditional Views created unique automation challenges — some elements are automatable via Appium, others require direct ADB MotionEvent injection.

Section 2: Roles & Responsibilities

> ***Resume-ready bullet points based on actual project work:***

Senior Mobile QA Automation Engineer — Superr (Ed-Tech)

• Designed and developed an enterprise-grade mobile automation framework from scratch using **Java, Appium 2.x, TestNG, and Maven** for the Superr Android tablet application, achieving comprehensive test coverage across 7 application modules.

• Architected a dual-mode **MobileDriverFactory** with a configurable `installApp` boolean flag enabling both fresh APK installation (`noReset=false`) and existing-app launch (`noReset=true`) flows, reducing test cycle time by eliminating unnecessary reinstalls during iterative development.

• Engineered a custom **HandwritingEngine** with a 73-character glyph map (a-z, A-Z, 0-9, punctuation) that programmatically writes text on the Canvas by computing stroke paths from normalized coordinates, enabling automation of handwriting-based features that standard Appium cannot interact with.

• Built a **StylusInjectorUtils** system that injects TOOL_TYPE_STYLUS MotionEvents via a companion APK and ADB instrumentation, bypassing the app's palm rejection logic that blocks standard Appium touch events (TOOL_TYPE_FINGER).

• Developed a **multi-type quiz automation flow** handling MCQ, Fill-in-Blank, and True/False questions within a single timed Speed Run session, with intelligent quiz-end detection using universal Check-button polling instead of question-type-specific text matching.

• Implemented robust **Page Object Model** architecture with 6+ page objects (LoginPage, MyNotesPage, QuizArenaPage, TextbookPage, AdminPortalPage, LMSPage), each following consistent try/catch + logging + ExtentReports patterns.

• Created a **ConfigReader** system supporting multi-environment configuration (dev/staging/prod) with device-specific property files for board and notebook tablet variants.

• Designed an **AI Tutor (SuperrChat) automation flow** covering Smart Markup → action selection → AI response wait → canvas-based follow-up chat → Chat History verification, handling asynchronous "Thinking" state detection with polling.

• Automated **Library class switching** with dynamic locator strategy using `textContains("CBSE, Class")` to handle variable class selections (Class 1 through Class 12) without hardcoded locators.

• Implemented comprehensive **scroll-based element discovery** using `scrollUntilElementAndClick()` for elements positioned beyond the viewport (CBSE filter badge, Class 12 in Library Selection, logout button).

• Developed a **canvas-aware quiz automation strategy** where Fill-in-Blank and True/False questions hidden behind canvas overlays are automated via HandwritingEngine with hardcoded answer sequences, while MCQ questions use coordinate-based radio selection via StylusInjector.

• Built a complete **web automation layer** using Selenium 4 for LMS Portal and Admin Portal, including multi-file upload testing, right-click context menu interactions, toast message validation, and browser-based login flows.

• Configured **ExtentReports** with TestListener integration for HTML test reports with pass/fail/info logging, automatic screenshot capture on failure, and step-by-step test execution tracking.

• Established **ADB utility layer** (AdbUtils) for device management including screen wake/sleep, unlock, home button, and the critical `setprop debug.input.simulate_stylus_with_touch true` property for stylus simulation.

• Created a hybrid **BaseTest architecture** with AndroidBaseTest, BaseTest, and HybridBaseTest supporting tablet-only, web-only, and cross-platform test scenarios from a single codebase.

• Developed a **quiz result validation system** that tracks correct/incorrect/skipped counters across mixed question types and verifies aggregate scores against the results screen, handling dynamic response messages from a pool of randomized feedback texts.

Section 3: Complete Framework Architecture

Architecture Overview

```
SuperrQAAutomation/
■■■ pom.xml ← Maven dependencies and build config
■
■■■ src/main/java/com/superr/framework/
■ ■■■ api/
■ ■ ■■■ AuthService.java ← API-based authentication
■ ■ ■■■ SessionService.java ← Session linking
■ ■ ■■■ GraphQLService.java ← GraphQL query support
```

```
■ ■
■ ■■■ base/
■ ■ ■■■ BasePage.java ← Android page base (By locators, click, getText)
■ ■ ■■■ AndroidBasePage.java ← Extended base with AndroidDriver specifics
■ ■
■ ■■■ driver/
■ ■ ■■■ DriverFactory.java ← Chrome/Edge WebDriver creation
■ ■ ■■■ DriverManager.java ← ThreadLocal for web + mobile drivers
■ ■ ■■■ MobileDriverFactory.java ← AndroidDriver creation (installApp flag)
■ ■
■ ■■■ listeners/
■ ■ ■■■ TestListener.java ← TestNG ITestListener → ExtentReports
■ ■
■ ■■■ pageobjects/
■ ■ ■■■ android/superrbook/
■ ■ ■ ■■■ common/LoginPage.java
■ ■ ■ ■■■ library/QuizArenaPage.java
■ ■ ■ ■■■ mynotes/MyNotesPage.java
■ ■ ■ ■■■ textbook/TextbookPage.java
■ ■ ■■■ web/
■ ■ ■■■ adminportal/StageAdminPortalPage.java
■ ■ ■■■ lmsportal/LMSPage.java
■ ■
■ ■■■ utils/
■ ■■■ AdbUtils.java ← Device ADB commands
■ ■■■ AndroidActions.java ← Swipe, scroll, tap helpers
■ ■■■ AppiumServerManager.java ← Auto-start Appium server
■ ■■■ ConfigReader.java ← Properties file loader
■ ■■■ ExtentManager.java ← Report singleton
■ ■■■ HandwritingEngine.java ← Canvas text writing (73 glyphs)
■ ■■■ LoggerUtils.java ← Log4j2 wrapper
■ ■■■ ReportUtils.java ← Extent step logging
■ ■■■ ScreenshotUtils.java ← Base64 screenshot capture
■ ■■■ StylusInjectorUtils.java ← TOOL_TYPE_STYLUS event injection
■ ■■■ WaitUtils.java ← Explicit wait helpers
■
■■■ src/test/java/com/superr/tests/
■ ■■■ base/
■ ■ ■■■ AndroidBaseTest.java ← Mobile test lifecycle
■ ■ ■■■ BaseTest.java ← Web test lifecycle
■ ■ ■■■ HybridBaseTest.java ← Cross-platform lifecycle
■ ■■■ android/superrbook/
■ ■■■ mynotes/MyNotes.java
■ ■■■ library/QuizArenaTests.java
■ ■■■ textbook/TextbookTests.java
■
■■■ src/main/resources/config/
■■■ devices/
■ ■■■ board_config.properties
■ ■■■ notebook_config.properties
■■■ environments/
■ ■■■ config-dev.properties
■ ■■■ config-staging.properties
■■■ testdata/apks/
■■■ appmainstylusdisabled.apk
■■■ stylusinjector.apk
```

Component-by-Component Justification

MobileDriverFactory — Why It Exists

Problem it solves: Appium driver creation has two fundamentally different modes — fresh install (with APK path) and existing-app launch (with package/activity). These require different UiAutomator2Options.

How it works:

```
boolean installMode = Boolean.parseBoolean(ConfigReader.get("installApp"));
if (installMode) {
options.setApp(ConfigReader.get("appPath")); // installs APK
// noReset = false (default) — clears app data
} else {
options.setAppPackage(ConfigReader.get("appPackage"));
options.setAppActivity(ConfigReader.get("appActivity"));
options.setNoReset(true); // preserves existing data
}
```

Why not just one mode? Fresh install is needed for CI/CD and first-time setup. Existing-app mode is needed for iterative test development — installing a 100MB APK before every test wastes 30-60 seconds.

HandwritingEngine — Why It Exists

Problem it solves: The Superr notebook is a Canvas-based custom view. Appium's `sendKeys()` doesn't work on Canvas — there's no text input field. `tap()` only places a dot. To write "IOT" or "big data" on the canvas, we need to draw character strokes.

How it works:

1. Each character ('A'-'Z', 'a'-'z', '0'-'9', punctuation) is defined as a set of normalized stroke paths in the GLYPHS map
2. Each stroke is a list of `double[]{x, y}` points normalized to 0.0-1.0 range
3. `writeText()` locates the canvas element, calculates pixel positions, and injects strokes via StylusInjectorUtils
4. Character spacing: `letterWidth` (default 18px) + `letterGap` (default 4px)

Why not just `element.sendKeys()`? Canvas elements don't accept keyboard input. They only respond to touch/stylus events. The HandwritingEngine translates text into physical stroke movements.

StylusInjectorUtils — Why It Exists

Problem it solves: The Superr app has **palm rejection** — it only accepts drawing input from a stylus (TOOL_TYPE_STYLUS), not finger touches (TOOL_TYPE_FINGER). Standard Appium touch actions send TOOL_TYPE_FINGER events, which the canvas ignores.

How it works:

1. Converts pixel coordinates to JSON array of {action, x, y, pressure, delay}
2. Pushes JSON to device via `adb push`
3. Runs `adb shell am instrument` to trigger the StylusInjector APK
4. The APK reads the JSON and injects real MotionEvents with TOOL_TYPE_STYLUS
5. Canvas receives stylus events → palm rejection passes → drawing appears

Why a separate APK? Android's `adb shell input` command only injects TOOL_TYPE_FINGER events. To inject TOOL_TYPE_STYLUS, you need a running Android app with the correct permissions to create MotionEvents with the stylus tool type. The StylusInjector APK is that bridge.

ConfigReader — Why It Exists

Problem it solves: Different environments (dev/staging/prod) have different URLs, credentials, and device configs. Hardcoding these in test code makes the framework non-portable.

How it works:

```
ConfigReader.load("dev"); // loads config-dev.properties
String url = ConfigReader.get("url");
String deviceId = ConfigReader.get("deviceId");
```

WaitUtils — Why It Exists

Problem it solves: Appium's implicit waits are unreliable for complex UI transitions (quiz loading, AI response, page navigation). Explicit waits with specific conditions are needed.

Key methods:

- `waitForElementVisible(driver, locator, timeout)` — polls until element is visible
- Used extensively for quiz-end detection, AI "Thinking" state, and page transitions

AndroidActions — Why It Exists

Problem it solves: Swiping, scrolling, and tapping at coordinates are common operations that need consistent implementation across page objects.

Key methods:

- `swipeUp()`, `swipeDown()`, `swipeLeft()`, `swipeRight()`
- `scrollUntilElementAndClick(locator, maxScrolls)` — scrolls until element found
- `tapOnCoordinates(x, y)` — coordinate-based taps for elements that can't be located normally

---

Section 4: Real Automation Challenges

Challenge 1: Canvas Elements Not Inspectable by Appium

Problem

Fill-in-Blank and True/False quiz questions are covered by a Canvas overlay. Appium Inspector cannot see the question text, question number, or question type label behind the canvas.

Root Cause

Android Canvas draws directly on a View surface. The drawn content has no accessibility nodes — Appium/UiAutomator2 can only see the View container, not what's drawn on it.

Symptoms

- Appium Inspector shows only the Canvas element with `content-desc="Canvas"` and bounds
- No child elements visible (no question text, no option text for FIB/T-F)
- Cannot verify which question is displayed
- Cannot determine question type (Fill-in-Blank vs True/False)

How We Debugged

1. Opened Appium Inspector on a Fill-in-Blank question
2. Compared the element tree with an MCQ question (which shows full text)

3. Confirmed: MCQ elements are standard TextViews — inspectable. FIB/T-F are behind Canvas — invisible.

Final Solution

Hardcoded question flow. Since question order and types are deterministic for a given quiz configuration (Chemistry → Electrochemistry → Batteries), we defined the action for each question upfront:

```
String[] questionActions = {"WRITE", "ANSWER", "WRITE", "ANSWER", "WRITE", "WRITE", "WRITE",
"SKIP"};
String[] writeAnswers = {"Sm-1", "", "F", "", "0 V", "T", "T", ""};
int[][] radioCoords = {{0,0}, {142,973}, {0,0}, {142,1006}, {0,0}, {0,0}, {0,0}, {0,0}};
```

Why This Worked

For a fixed quiz configuration, the question order is predictable. The test doesn't need to read the question — it just executes the predetermined action for each position.

Interview Explanation

"In the Superr quiz, Fill-in-Blank and True/False questions render behind a Canvas overlay, making them completely invisible to Appium Inspector. We couldn't detect question type or text at runtime. Our solution was a data-driven hardcoded approach — since the quiz configuration (subject/chapter/topic) determines question order, we pre-mapped each question's action type and answer. For FIB, we used our custom HandwritingEngine to write answers on the canvas. For MCQ, we used coordinate-based radio selection."

Prevention

Request developers to add accessibility labels to Canvas-based questions, or expose question metadata via `content-desc` attributes on the Canvas container.

Challenge 2: Stylus vs Finger — Palm Rejection Blocking Appium Touch Events

Problem

Standard Appium touch/tap actions were being IGNORED by the canvas. Taps, drags, and swipes on the notebook canvas produced no visible result.

Root Cause

The Superr app implements **palm rejection** — a feature that filters input by `MotionEvent.getToolType()`. Only TOOL_TYPE_STYLUS (value 2) events are accepted for drawing on canvas. Appium generates TOOL_TYPE_FINGER (value 1) events, which the palm rejection filter rejects.

Symptoms

- `locator.click()` on canvas → nothing happens
- `Actions().moveToElement().click()` → nothing happens
- Manual finger touch on real device → nothing happens (need physical stylus)
- But `adb shell input tap x y` → also nothing (also TOOL_TYPE_FINGER)

How We Debugged

1. Confirmed that physical stylus input works on real device

2. Tested `adb shell input tap` — also rejected
3. Researched Android MotionEvent tool types
4. Found that `adb shell input` always generates TOOL_TYPE_FINGER
5. Identified that we need TOOL_TYPE_STYLUS at the MotionEvent level

Final Solution

Built a **StylusInjector companion APK** that:

1. Receives stroke coordinates via JSON file pushed to device
2. Creates real MotionEvents with TOOL_TYPE_STYLUS
3. Injects them into the system via Android Instrumentation
4. Canvas receives stylus events → palm rejection passes → drawing appears

```
// StylusInjectorUtils.injectStroke flow:
// 1. Build JSON: [{action:0, x:400, y:600, pressure:0.45, delay:20}, ...]
// 2. adb push json to /data/local/tmp/stylus_stroke.json
// 3. adb shell am instrument -w com.superr.stylusinjector/.StylusInjector
// 4. StylusInjector APK reads JSON → injects MotionEvents with TOOL_TYPE_STYLUS
```

Why This Worked

The StylusInjector APK runs ON the device as an Android app, so it has full access to the MotionEvent API. It can create events with any tool type, including TOOL_TYPE_STYLUS, which bypasses palm rejection.

Interview Explanation

"The biggest challenge was that the app uses palm rejection — it only accepts stylus input (TOOL_TYPE_STYLUS), not finger touches (TOOL_TYPE_FINGER). Since Appium and ADB both generate finger events, our canvas interactions were silently rejected. We built a companion StylusInjector APK that runs as an Android Instrumentation test on the device. It reads stroke coordinates from a JSON file and injects real MotionEvents with TOOL_TYPE_STYLUS. This was the only way to automate canvas writing — no standard Appium approach could solve it."

What Developers Changed

Developers provided a separate build variant `appmainstylusdisabled.apk` where palm rejection is disabled for testing. However, we still use the StylusInjector for production-like testing to validate the actual stylus workflow.

Challenge 3: Quiz-End Detection for Multi-Type Quizzes

Problem

The existing `isQuizEndedAfterNext()` method only checked for "MCQ" text visibility to detect if a new question loaded. When the next question was Fill-in-Blank or True/False (no "MCQ" text), the detection failed.

Root Cause

The detection method was designed for MCQ-only quizzes. It used `mcqText` ("MCQ") as the positive signal that a new question loaded. FIB questions show "Fill the Blank" text, and T/F questions show "True or False" — neither matches "MCQ".

Symptoms

- Test hangs after clicking Next on a MCQ question followed by a FIB question
- 15-second timeout → test assumes quiz ended early when it actually hasn't
- False quiz-end detection mid-quiz

How We Debugged

1. Ran the multi-type quiz manually and noted question type transitions
2. Identified that "MCQ" text only appears on MCQ questions
3. Confirmed that FIB/T-F questions show their type labels behind canvas (not inspectable)
4. Needed a UNIVERSAL signal that works for ALL question types

Final Solution

Created `isMultiTypeQuizEndedAfterAction()` that uses the **Check button** (`content-desc="Check"`) as the universal detection signal:

- After clicking Next, the Check button disappears during transition
- When the new question loads (ANY type), Check reappears
- If quiz ends instead, transition messages or results title appear

```
private boolean isMultiTypeQuizEndedAfterAction() {
long deadline = System.currentTimeMillis() + 15000;
while (System.currentTimeMillis() < deadline) {
// Priority 1: Check button visible → next question loaded (UNIVERSAL)
if (driver.findElements(checkBtnId).size() > 0) return false;
// Priority 2: Transition messages → quiz ending
if (isAnyTransitionVisible()) { waitForResultsTitle(30); return true; }
// Priority 3: Results title → quiz ended
if (isResultsTitleVisible()) return true;
pause(200);
}
return isResultsTitleVisible();
}
```

Interview Explanation

"Our quiz had three question types — MCQ, Fill-in-Blank, and True/False. The existing detection only checked for 'MCQ' text to confirm the next question loaded. When the next question was FIB or T/F, this failed. We redesigned the detection to use the Check button (accessibility ID) as a universal signal — it appears on ALL question types and disappears during transitions. This made the detection type-agnostic."

Challenge 4: Writing Answers on the AI Tutor Chat Canvas

Problem

The AI Tutor chat panel has a "write here" canvas area at the bottom. The Send button only appears AFTER writing on this canvas. The canvas had no `content-desc` or accessibility ID.

Root Cause

The canvas is a generic `android.view.View` without semantic attributes. It's identified only by its bounds `[0,1384][1200,1920]` in the XML hierarchy.

Symptoms

- Cannot find the canvas element via accessibility ID or content-desc
- `page.getByAccessibilityId("Canvas")` fails — no matching element
- Send button invisible until something is drawn on the canvas

How We Debugged

1. Used Appium Inspector to examine the AI Tutor panel
2. Found the canvas View with bounds `[0,1384][1200,1920]` but no content-desc
3. Confirmed Send button only appears post-drawing

Final Solution

Used a **bounds-based XPath** locator as a temporary solution:

```
private By aiTutorChatCanvas =
AppiumBy.xpath("//android.view.View[@bounds='[0,1384][1200,1920]']");
// TODO: Switch when dev adds accessibility ID:
// private By aiTutorChatCanvasById = AppiumBy.accessibilityId("Canvas");
```

Then used HandwritingEngine to write on it. Filed a request with developers to add `content-desc="Canvas"` to the element.

Challenge 5: Dynamic CBSE Class Filter Badge

Problem

The Library page has a "CBSE, Class X" badge at the bottom that shows the currently selected class. The class number varies per user/session — could be "CBSE, Class 1" through "CBSE, Class 12".

Root Cause

The badge text is dynamic — it reflects the user's current class selection, which can be any of 10 values.

Symptoms

- Hardcoding `text("CBSE, Class 12")` fails when the user's class is different
- Cannot predict which class will be shown

Final Solution

Used `textContains()` to match ANY class:

```
private By cbseClassFilterBtn = AppiumBy.androidUIAutomator(
"new UiSelector().textContains(\"CBSE, Class\")"
);
```

This matches "CBSE, Class 1", "CBSE, Class 5", "CBSE, Class 12" — all variants.

Interview Explanation

"We had a dynamic badge text that changed based on the user's selected class. Instead of predicting the value, we used `textContains('CBSE, Class')` which matches any variant. This is a common pattern for dynamic text — match the static part, ignore the variable part."

Challenge 6: AI Tutor "Thinking" State Detection

Problem

After clicking "Explain this" or sending a message, the AI Tutor shows "Thinking" while processing. The test needs to wait for the AI to finish before interacting with the response.

Root Cause

AI response time is variable (2-30+ seconds). No fixed wait works.

Symptoms

- Test tries to write on canvas while AI is still generating
- Canvas might not accept input during "Thinking" state
- `Thread.sleep()` either waits too long (slow) or not enough (flaky)

Final Solution

Two-phase polling approach:

1. **Phase 1:** Wait for "Thinking" text to APPEAR (confirms AI started)
2. **Phase 2:** Poll until "Thinking" text DISAPPEARS (confirms AI finished, up to 60s)

```
public void waitForThinking() {
// Phase 1: Thinking appears
WaitUtils.waitForElementVisible(driver, thinkingText, 30);
// Phase 2: Thinking disappears
long deadline = System.currentTimeMillis() + 60000;
while (System.currentTimeMillis() < deadline) {
if (driver.findElements(thinkingText).size() == 0) {
pause(1000); return;
}
pause(1500);
}
}
```

Challenge 7: MCQ Radio Button Selection via Coordinates

Problem

MCQ radio buttons in the quiz don't have unique `content-desc` or `resource-id`. They're rendered as generic Views.

Root Cause

The quiz renders options as custom views without accessibility identifiers. Standard locators cannot distinguish between Option A, B, C, D.

Final Solution

Used **coordinate-based taps** via StylusInjectorUtils:

```
public void selectRadioViaStylusInjector(int x, int y) {
StylusInjectorUtils.injectStroke(deviceId, new int[][]{{x, y}, {x, y+1}}, 40);
}
```

Coordinates were determined by inspecting each option's bounds in the XML dump.

Challenge 8: Leave Quiz Dialog — Close Button Ambiguity

Problem

The quiz screen has a close (X) button that triggers the "Leave Quiz?" dialog. The results screen ALSO has a close button with `accessibilityId("Close")`. How to distinguish between them?

Root Cause

Both close buttons share the same accessibility ID "Close". In different states (during quiz vs on results), they serve different purposes.

Final Solution

Context-aware usage — the test knows its state:

- During quiz: clicking `Close` triggers the Leave Quiz dialog
- On results: clicking `Close` exits the results screen

The same locator is reused because the test explicitly knows when it's in the quiz vs results context.

Section 5: Superr Application-Specific Challenges

These problems exist ONLY because of Superr's unique architecture:

Challenge	Why Superr-Specific	Why Standard Appium Can't Solve It	Our Workaround
Palm rejection blocks all Appium touch events	Superr uses stylus-only canvas	Appium generates TOOL_TYPE_FINGER	StylusInjector APK with TOOL_TYPE_STYLUS
Canvas hides quiz question text	Superr renders FIB/T-F on Canvas	UiAutomator2 can't see Canvas content	Hardcoded question actions per quiz config
Handwriting recognition input	Superr expects written text, not typed	No `sendKeys()` on Canvas	HandwritingEngine with 73-character glyph map
AI Tutor variable response time	SuperrChat uses AI (variable latency)	Fixed waits don't work	Two-phase Thinking polling
Send button only visible after drawing	AI chat canvas input triggers Send visibility	Can't click invisible element	Draw first (HandwritingEngine), then wait for Send
Pen palette tool selection	Custom pen palette with tool slots	Not standard Android spinner/dropdown	Coordinate-based selection + scroll
Page indicator canvas	Quick Notes page count on Canvas	Can't read drawn text	`getPageIndicatorText()` reads accessible text
Quiz response message pool	Randomized feedback ("Boom!", "Oops!")	Can't predict exact message	Pattern matching against known message arrays

---

Section 6: Developer Discussions

Discussion 1: Palm Rejection for Automation

Problem: Appium touch events completely ignored by Canvas.

Developer Explanation: The app uses `MotionEvent.getToolType()` to filter input. Only TOOL_TYPE_STYLUS passes the filter. This is by design — prevents accidental palm touches while writing.

Developer Fix: Provided a separate APK build variant `appmainstylusdisabled.apk` with palm rejection disabled. Also set `debug.input.simulate_stylus_with_touch true` ADB property.

Automation Impact: We use the stylus-disabled APK for basic tests and the StylusInjector for production-like stylus tests.

Discussion 2: Missing Accessibility Identifiers on Canvas

Problem: AI Tutor chat canvas has no `content-desc` or `resource-id`.

Developer Explanation: The canvas is a custom View created dynamically — accessibility attributes weren't added.

Developer Fix: Agreed to add `content-desc="Canvas"` in a future build.

Automation Impact: Using bounds-based XPath as a temporary locator until the fix ships. Commented-out accessibility ID locator is ready to swap.

Discussion 3: Quiz Question Type Labels Behind Canvas

Problem: FIB and T-F question type labels ("Fill the Blank", "True or False") are invisible to Appium.

Developer Explanation: The Canvas view is drawn on top of the question label — it's a rendering order issue, not intentional.

Automation Impact: We cannot dynamically detect question type at runtime. Using hardcoded action sequences per quiz configuration.

Section 7: Locator Challenges

Locator Strategy Summary

Locator Type	When Used	Example	Performance
`accessibilityId` (content-desc)	Primary — most reliable, fastest	`AppiumBy.accessibilityId("SuperrChat")`	■ Best
`androidUIAutomator` (UiSelector)	Secondary — for text, textContains	`new UiSelector().text("Chemistry")`	■ Very Good
`androidUIAutomator` (textContains)	Dynamic text	`new UiSelector().textContains("CBSE, Class")`	■ Good
`androidUIAutomator` (instance)	Multiple same-desc elements	`new UiSelector().description("Close").instance(1)`	■ Good

XPath (bounds-based)	Canvas without accessibility ID	`//android.view.View[@bounds='[0,1384][1200,1920]']`	■■ Fragile
Coordinate-based	Canvas elements, radio buttons	`StylusInjectorUtils.injectStroke(id, {{142,973}}, 40)`	■■ Device-specific

Specific Locator Challenges

Resource-ID Missing

Most Superr UI elements lack `resource-id`. The app uses custom views without standard Android IDs.

Workaround: Heavy reliance on `content-desc` (accessibility ID) and `text` attributes.

Dynamic Locators for Quiz Responses

Quiz feedback messages are randomized from a pool ("That's spot on!", "Boom, that's right!", "Oops, that's wrong").

Solution: Pattern matching against known arrays:

```
private static final String[] CORRECT_MESSAGES = {"That's spot on!", "Boom, that's right!",
"Absolutely perfect!"};
```

Canvas Elements — Zero Inspectable Content

Canvas-drawn content has no accessibility tree nodes.

Solution: Bypass inspection entirely — use HandwritingEngine to write and StylusInjector to interact.

Close Button Instance Disambiguation

Multiple "Close" buttons on screen (notebook close + AI panel close).

Solution: `new UiSelector().description("Close").instance(1)` — second instance is the AI panel.

Section 8: Jetpack Compose Analysis

Does Superr Use Jetpack Compose?

No. Based on all evidence from our automation work:

1. **Element hierarchy:** Inspector shows traditional `android.view.View`, `android.widget.TextView`, `android.widget.ImageView` — NOT `ComposeView` or Compose semantic nodes
2. **Locator strategy:** All locators use `UiSelector().text()`, `UiSelector().description()`, standard XPath — these are traditional Android View patterns
3. **No Compose test tags:** We never encountered `testTag` attributes or Compose-specific semantics
4. **Inspector behavior:** Elements have standard `bounds`, `text`, `content-desc` attributes — Compose elements render differently in the hierarchy

How to Identify Compose vs Traditional Views

Indicator	Traditional Views	Jetpack Compose
Root elements	`android.widget.FrameLayout`, `android.view.View`	`android.view.View` wrapping `ComposeView`
Text elements	`android.widget.TextView` with `text` attr	Semantic node with `Text` role
Locator strategy	`UiSelector().text()`, `accessibilityId()`	`testTag()` via Compose semantics
Inspector depth	Deep nested hierarchy	Flatter structure with semantic grouping

Automation Impact

Since Superr uses traditional Android Views (not Compose), standard UiAutomator2 locators work. Had it been Compose, we would have needed `testTag`-based locator strategies and potentially different Inspector tooling.

Section 9: Native vs Hybrid vs WebView Analysis

Verdict: Native Android with Canvas Custom Views

Type	Present?	Evidence
Native Android	■ Yes (primary)	Standard Android Views in Inspector: `android.view.View`, `android.widget.TextView`, `android.widget.ImageView`
Canvas Custom Views	■ Yes (significant)	Notebook, quiz question overlay, AI chat input — all Canvas-based
Hybrid	■ No	No WebView containers in Inspector hierarchy
WebView	■ No	No `android.webkit.WebView` observed
Jetpack Compose	■ No	No `ComposeView` or Compose semantics

How We Identified the Architecture

1. **Appium Inspector examination:** Every screen we automated showed `android.view.View` and `android.widget.TextView` elements — native Android.
2. **Canvas detection:** When inspecting the notebook and quiz screens, we found generic `android.view.View` elements with no children where content should be — indicating Canvas rendering.
3. **No WebView context switching:** We never needed `driver.switchTo().context()` — a telltale sign of Hybrid/WebView apps.
4. **ADB diagnostics:** `adb shell dumpsys activity` showed native activities, not WebView-based ones.

Automation Impact

Architecture Aspect	Impact
Native Views	Standard Appium locators work (accessibilityId, UiSelector, XPath)
Canvas Views	Appium CANNOT interact → StylusInjector + HandwritingEngine required
No WebView	No context switching needed, simplifies driver management
Mixed (Native + Canvas)	Some elements automatable normally, some require custom solutions

---

Section 10: Interview Questions & Answers (50+)

Architecture & Framework

Q1: Tell me about the application you automated.

Superr is an educational tablet app for students. It has a notebook feature where students write with a stylus on a Canvas, a textbook library, a quiz engine with MCQ/Fill-in-Blank/True-or-False, and an AI tutor. The unique challenge is that it's a Canvas-based native Android app — meaning large portions of the UI are rendered on Canvas and can't be inspected by standard Appium tools.

Q2: Describe your automation framework architecture.

Java + Appium 2.x + TestNG + Maven. Page Object Model with separate page classes for each screen. MobileDriverFactory with an installApp boolean flag — true installs a fresh APK each run, false launches the existing app. Custom utilities for stylus injection (StylusInjectorUtils), handwriting (HandwritingEngine), and device management (AdbUtils). ExtentReports for HTML reporting with automatic screenshot on failure.

Q3: Why did you build a custom HandwritingEngine?

Because the app uses Canvas for student handwriting. Appium's sendKeys doesn't work on Canvas — there's no text field. We needed to physically draw each character stroke-by-stroke. The engine has 73 character definitions with normalized coordinates. It calculates pixel positions from the canvas bounds and injects strokes via StylusInjector with TOOL_TYPE_STYLUS to bypass palm rejection.

Q4: Explain the installApp flag.

It's a boolean in our config file. `true` means install the APK fresh every run — slower but gives a clean state, used in CI. `false` means launch the already-installed app — faster, used during development. The MobileDriverFactory reads this flag and sets different UiAutomator2Options: `setApp(path)` for install mode, `setAppPackage/setAppActivity` for launch mode.

Q5: How do you handle parallel execution?

ThreadLocal driver management via DriverManager ensures each test thread has its own AndroidDriver instance. Each test gets its own device connection. TestNG's parallel configuration handles thread allocation.

Challenges & Debugging

Q6: What was the biggest challenge in your Appium automation?

Palm rejection. The app only accepts TOOL_TYPE_STYLUS input on Canvas, but Appium generates TOOL_TYPE_FINGER events. Every Canvas interaction was silently rejected. We built a StylusInjector companion APK that runs on the device and injects real MotionEvents with the correct tool type. This was a completely custom solution — no standard Appium approach could solve it.

Q7: Tell me about a debugging experience.

When automating the multi-type quiz, our quiz-end detection kept falsely reporting that the quiz ended after each question. After debugging, we found the detection method checked for 'MCQ' text to confirm the next question loaded. But Fill-in-Blank questions don't show 'MCQ' text — the detection failed. We redesigned it to use the Check button (accessibility ID) as a universal signal that works for all question types.

Q8: How did you handle elements that Appium can't see?

For Canvas-hidden elements like Fill-in-Blank question text, we used a hardcoded approach. Since the quiz configuration determines question order, we pre-mapped each question's action and answer. For Canvas interaction, we used our HandwritingEngine + StylusInjector. For elements without accessibility IDs, we used bounds-based XPath as a temporary measure and filed requests with developers.

Q9: Tell me about a blocker and how you resolved it.

The AI Tutor's Send button was invisible until the user writes on the canvas. We couldn't click it first. The canvas itself had no accessibility ID — just bounds. Solution: write on the canvas first using HandwritingEngine (which triggers Send to appear), then click Send. For the canvas locator, we used bounds-based XPath and requested developers add a proper content-desc.

Q10: How did you handle flaky tests?

Multiple strategies: (1) Robust waits — our waitForThinking() polls for the Thinking text to appear AND disappear instead of using fixed sleeps. (2) Generous timeouts on quiz transitions — 15-second polling for next-question detection. (3) Scroll-before-click patterns — isDisplayed check before scrolling, to handle variable viewport positions.

Locators & Strategy

Q11: What locator strategy do you use?

Priority order: accessibilityId first (fastest, most stable), then UiSelector with text/textContains for dynamic elements, then UiSelector with instance() for disambiguating multiple same-named elements. XPath only as last resort — we have one bounds-based XPath for the AI chat canvas. Coordinates for Canvas elements where no locator is possible.

Q12: Why didn't XPath work well for you?

XPath is slow on Android because UiAutomator2 has to traverse the entire view hierarchy. More importantly, Canvas elements have no child nodes in the hierarchy — XPath can't find what's not there. We avoided XPath wherever possible in favor of accessibilityId and UiSelector.

Q13: How did you handle dynamic locators?

textContains for the CBSE badge (varies Class 1-12), pattern matching against known message arrays for quiz feedback, and instance(N) for disambiguating multiple Close buttons.

Quiz Automation

Q14: How did you automate a mixed-question quiz?

We created a multiTypeQuizFlow method that accepts three arrays: question actions (WRITE/ANSWER/SKIP), write answers for canvas questions, and radio coordinates for MCQ questions. For WRITE: HandwritingEngine writes on canvas → Check → track response → Next. For ANSWER: StylusInjector taps radio → Check → track → Next. For SKIP: direct skip. Universal quiz-end detection using Check button polling.

Q15: How did you validate quiz results?

We tracked correct/incorrect/skipped counters during the quiz by matching response messages against known arrays (CORRECT_MESSAGES, INCORRECT_MESSAGES). After the quiz, we verified the counters against the results screen's displayed scores.

Q16: How do you handle the quiz timer?

The Speed Run has a countdown timer (5/10/20 minutes). Our quiz-end detection method handles early termination — if the timer expires mid-quiz, transition messages appear and we detect them. The test gracefully handles partial quiz completion.

Canvas & Stylus

Q17: How does your HandwritingEngine work?

Each character is defined as normalized stroke paths (0.0-1.0 coordinates). writeText() locates the canvas element, calculates pixel positions based on canvas bounds, and calls StylusInjectorUtils.injectGlyphStroke() for each character's strokes. The engine handles letter spacing (18px width + 4px gap), positioning (CENTER, TOP_LEFT), and multi-line text.

Q18: What is TOOL_TYPE_STYLUS vs TOOL_TYPE_FINGER?

Android MotionEvents have a tool type field. TOOL_TYPE_FINGER (1) is for finger touches — what Appium and ADB generate. TOOL_TYPE_STYLUS (2) is for stylus/pen input. Apps with palm rejection check this field and reject TOOL_TYPE_FINGER on canvas areas. Our StylusInjector APK creates MotionEvents with TOOL_TYPE_STYLUS.

Q19: How does the StylusInjector APK work?

It's an Android Instrumentation test APK. We push a JSON file with stroke coordinates to the device, then run `adb shell am instrument` to trigger the APK. It reads the JSON, creates MotionEvents with ACTION_DOWN/MOVE/UP, sets TOOL_TYPE_STYLUS, applies natural pressure curves, and injects them into the input system. The canvas sees them as real stylus strokes.

AI Tutor

Q20: How did you automate AI-powered features?

The AI Tutor shows 'Thinking' while processing. We poll for the Thinking text to appear (confirms AI started), then poll until it disappears (confirms response loaded). Between these two phases, we don't interact with the UI. After the response loads, we write follow-up questions on the canvas and send them.

Web Automation

Q21: How did you handle web + mobile in the same framework?

Three base test classes: AndroidBaseTest for tablet, BaseTest for web (Chrome), HybridBaseTest for cross-platform flows. DriverManager uses ThreadLocal to store both WebDriver and AndroidDriver. Each base test class initializes the appropriate driver type.

Q22: How did you handle file uploads in LMS Portal?

Standard Selenium sendKeys on the file input element. For multiple files, we pass an array of file paths. For file deletion, we use right-click (context menu) via Selenium's Actions class.

Configuration & DevOps

Q23: How do you manage test data across environments?

Properties files per environment: config-dev.properties, config-staging.properties. ConfigReader.load(env) loads the right file. Credentials, URLs, and device configs are all externalized. The env is passed via Maven's -Denv=staging parameter.

Q24: How do you handle APK management?

Two APKs: the main app APK and the StylusInjector APK. Both are in src/main/resources/config/testdata/apks/. The installApp flag controls whether the main APK is installed fresh or launched from existing installation. StylusInjector is installed via ADB before canvas tests.

Q25: How does your reporting work?

ExtentReports with a TestNG TestListener. Every test method logs pass/fail/info to the Extent test. Screenshots are captured on failure as Base64 and attached to the report. The HTML report shows step-by-step execution with timestamps.

Advanced Scenarios

Q26: How did you handle scroll to find elements?

scrollUntilElementAndClick(locator, maxScrolls) — scrolls down, checks if element is visible, clicks if found, repeats up to maxScrolls times. Used for CBSE filter badge at bottom of Library, Class 12 at bottom of selection list, and logout button.

Q27: How do you handle multiple instances of the same element?

UiSelector with instance(N). For example, the notebook Close button is instance(0) and the AI panel Close button is instance(1): `new UiSelector().description(\"Close\").instance(1)`.

Q28: Describe a situation where you had to use coordinate-based automation.

MCQ radio buttons in the quiz. They're rendered as generic Views without unique identifiers. We inspected the XML to get each option's bounds, calculated center coordinates, and used StylusInjectorUtils.injectStroke to tap at those coordinates.

Q29: How did you handle the Leave Quiz flow?

Click the cross button during the quiz → verify the 'Leave Quiz?' dialog title and 'You will lose all progress.' message via getText → click the Leave button (accessibility ID 'Leave Quiz') → verify redirect back to Speed Run config screen by checking for the Speed Run description text.

Q30: What's the difference between your three Base Test classes?

AndroidBaseTest initializes MobileDriverFactory, creates AndroidDriver, manages Appium server lifecycle. BaseTest initializes DriverFactory for Chrome/Edge WebDriver. HybridBaseTest initializes BOTH for cross-platform flows like LMS web + tablet verification.

Design Decisions

Q31: Why TestNG over JUnit?

TestNG's test suite XML files allow flexible test grouping (smoke, regression), priority-based execution, and data providers. The @Listeners annotation integrates cleanly with ExtentReports. Groups allow

running smoke tests on every build and full regression nightly.

Q32: Why Page Object Model?

Separation of concerns — locators and actions live in page classes, test logic lives in test classes. When a UI element changes, we update one page class, not every test. With 6+ page objects and 20+ test methods, this separation prevents maintenance chaos.

Q33: Why not use Espresso instead of Appium?

Espresso requires access to the app source code and runs inside the app process. We're a QA team testing a production APK — we don't have source code access. Appium works with any APK, black-box style.

Q34: Why separate device config files?

The Superr Book Tablet comes in two variants — board and notebook — with different hardware specs and screen configurations. Separate config files let us switch between device types without changing test code.

Bonus: Production & Process

Q35: How did you validate production builds?

Smoke test suite runs against every new APK build. Covers critical flows: login, notebook write, quiz start, AI tutor invoke. Uses installApp=true to test the actual APK that will ship to students.

Q36: How do you handle regression testing?

Full test suite with all quiz modes, notebook operations, library navigation, and AI tutor flows. Runs nightly or before releases. Results are tracked in ExtentReports with historical comparison.

Q37: Describe your bug reporting process.

When automation catches a bug: capture screenshot + trace from ExtentReport → document reproduction steps from test logs → file with expected vs actual from assertion → include the test method name and config for reproducibility.

Q38: How do you collaborate with developers?

Regular sync on accessibility improvements (adding content-desc to canvas elements), APK signing requirements, build variant management (stylus-disabled vs production), and understanding new feature architecture before writing automation.

Q39: What would you improve in the framework?

1) Migrate web tests to Playwright for auto-waiting and better debugging. 2) Add visual regression for Canvas content using screenshot comparison. 3) Implement storageState/session reuse to skip login in every test. 4) Add retry logic for flaky Canvas interactions. 5) CI/CD pipeline integration with Jenkins/GitHub Actions.

Q40: How do you handle test data cleanup?

installApp=true mode clears all app data (noReset=false). For installApp=false mode, tests are designed to be idempotent — quiz configurations are reusable, notebook writes don't depend on prior state.

Scenario-Based Questions

Q41: A test that passed yesterday is failing today. How do you debug?

1) Check the ExtentReport for the failure screenshot and logs. 2) Check if the APK version changed (new build might have UI changes). 3) Check if the device state is different (logged-in user, class selection). 4) Run the test in headed mode with logging. 5) Use Appium Inspector to verify element locators still match.

Q42: The test hangs on a quiz question. What do you check?

1) Is the quiz-end detection method timing out? Check if the 'Check' button or transition messages are present. 2) Has the timer expired? The quiz might have ended. 3) Is there an unexpected dialog (permission popup, network error)? 4) Is the element locator still valid? The question type might have changed.

Q43: Appium can't find an element that's visible on screen. Why?

1) It's behind a Canvas overlay (our FIB/T-F issue). 2) It's in a different context (WebView). 3) The accessibility tree doesn't include it (custom view without accessibility support). 4) The element is outside the current viewport (needs scroll). 5) The locator is wrong (text changed, accessibility ID changed).

Q44: How would you automate a feature with no testable identifiers?

1) Request developers add data-testid/content-desc. 2) Use bounds-based XPath as temporary solution. 3) Use coordinate-based taps as last resort. 4) Use relative locators (sibling/parent relationships). Always document the limitation and track the accessibility improvement request.

Q45: Describe a time when your automation found a real bug.

Our quiz result validation caught a mismatch where the results screen showed different correct/incorrect counts than what we tracked during the quiz. The root cause was a backend scoring issue where skipped questions were sometimes counted as incorrect.

Technical Deep-Dives

Q46: How does MotionEvent injection work at the Android level?

An Android app with Instrumentation permissions can create MotionEvent objects with any toolType, action (DOWN/MOVE/UP), coordinates, and pressure. The StylusInjector APK creates these events and injects them via `Instrumentation.sendPointerSync()`. The system dispatches them to the foreground window as if a real stylus touched the screen.

Q47: Why does UiAutomator2 use accessibility tree instead of the View hierarchy?

UiAutomator2 queries the AccessibilityNodeInfo tree, which is a flattened representation of the view hierarchy that includes only accessibility-relevant information. This is why Canvas content (which has no accessibility nodes) is invisible. It's also why custom Views without accessibility support are unmatchable.

Q48: How do you handle state-dependent test flows?

The multiTypeQuizFlow method is state-aware — it tracks correct/incorrect/skipped counts internally and passes them to the results verification method. For quiz-end detection, we use priority-based polling (Check button → transition messages → results title) to handle any possible transition.

Q49: What's the performance impact of coordinate-based locators?

Coordinate-based locators (tapOnCoordinates, radio selection) are fast but fragile — they break on different screen resolutions or DPI. Our framework is calibrated for the Superr Book Tablet's 1200×1920 resolution. For other devices, coordinates would need recalculation.

Q50: How would you scale this framework to support multiple devices in parallel?

1) Parameterize device configs (deviceId, deviceName) per thread. 2) Use TestNG's parallel='tests' with separate XML entries per device. 3) Each thread gets its own AndroidDriver via ThreadLocal. 4) Appium

server per device (different ports). 5) Coordinate-based locators would need device-specific scaling factors.

Q51: What metrics do you track for automation quality?

1) Test pass rate per suite (smoke, regression). 2) Flaky test rate (tests that alternate pass/fail). 3) Average test execution time. 4) Code coverage by module (which features have automation). 5) Bug detection rate (bugs found by automation before manual testing).

Q52: How do you keep locators maintainable?

1) Page Object Model — locators are centralized in page classes. 2) Naming convention — locator variables describe the element, not the locator type: `leaveQuizTitle` not `leaveQuizTextXpath`. 3) Priority: accessibilityId > text > textContains > bounds. 4) Comments with XML evidence for unusual locators.

> *This document is based entirely on real project work from the Superr QA automation framework development. Every challenge, solution, and code example comes from actual implementation and debugging sessions.*